

## Research Article

# A Reconfigurable Coarse-to-Fine Approach for the Execution of CNN Inference Models in Low-Power Edge Devices

Auangkun Rangsikunpum , Sam Amiri , and Luciano Ost 

*Wolfson School of Mechanical, Electrical and Manufacturing Engineering, Loughborough University, Loughborough, UK*

Correspondence should be addressed to Auangkun Rangsikunpum; [a.rangsikunpum@lboro.ac.uk](mailto:a.rangsikunpum@lboro.ac.uk)

Received 10 December 2023; Revised 2 March 2024; Accepted 28 March 2024

Academic Editor: Sudarshan K. Srinivasan

Copyright © 2024 Auangkun Rangsikunpum et al. This is an open access article distributed under the Creative Commons Attribution License, which permits unrestricted use, distribution, and reproduction in any medium, provided the original work is properly cited.

Convolutional neural networks (CNNs) have evolved into essential components for a wide range of embedded applications due to their outstanding efficiency and performance. To efficiently deploy CNN inference models on resource-constrained edge devices, field programmable gate arrays (FPGAs) have become a viable processing solution because of their unique hardware characteristics, enabling flexibility, parallel computation and low-power consumption. In this regard, this work proposes an FPGA-based dynamic reconfigurable coarse-to-fine (C2F) inference of CNN models, aiming to increase power efficiency and flexibility. The proposed C2F approach first coarsely classifies related input images into superclasses and then selects the appropriate fine model(s) to recognise and classify the input images according to their bespoke categories. Furthermore, the proposed architecture can be reprogrammed to the original model using partial reconfiguration (PR) in case the typical classification is required. To efficiently utilise different fine models on low-cost FPGAs with area minimisation, ZyCAP-based PR is adopted. Results show that our approach significantly improves the classification process when object identification of only one coarse category of interest is needed. This approach can reduce energy consumption and inference time by up to 27.2% and 13.2%, respectively, which can greatly benefit resource-constrained applications.

## 1. Introduction

Convolutional neural network (CNN) is one of the most popular deep learning techniques for researchers [1] mainly due to its excellent achievement on computer vision applications [2]. CNN models can extract complicated image features, showing remarkable success in various advanced applications, including image classification [3], object detection [4] and face recognition [5]. The CNN inference involves multiply-accumulate (MAC) operations, requiring reasonable computation time and resources (e.g. memory and power budget). CNN inference models can be deployed on robust cloud infrastructures or resource-constrained edge devices [6]. Although inference on the cloud can achieve higher performance from the deeper model due to the massive availability of resources from various locations, inference on edge devices is increasingly demanded in low-latency and high-privacy applications [7]. Inference on edge can be significantly beneficial in real-time applications, since edge devices are installed near the

source and can act rapidly if any response action is required. Additionally, the data are securely stored with the user and do not have to transfer to the cloud for processing.

Field programmable gate arrays (FPGAs) are suitable platforms for hardware implementation of CNN inference models [8, 9] due to their flexibility and parallel processing capabilities, enabling customised designs that align with the computational characteristics of CNNs (e.g. convolutional filters). Furthermore, FPGA-based CNN implementations are well-known for their ability to achieve notably lower power consumption w.r.t. graphics processing unit (GPU)-based solutions [10–12]. FPGA-based CNN designs can also benefit from the partial reconfiguration (PR). PR allows some part of the FPGA fabric (the dynamic part) to be reconfigured, while the other part (the static part) is still running. The defined dynamic portion can be modified by downloading a partial bitstream file, which is naturally smaller than the entire bitstream file. This flexibility enhancement feature is promising for implementing deep CNN architectures on a small

FPGA since parts of the CNN model can be loaded from the external memory to reprogram the dynamic part at runtime.

The above characteristics make FPGAs beneficial for deploying more efficient CNN solutions in resource-constrained systems, such as drones. Unmanned aerial vehicles (UAVs), also known as drones, are employed in different sectors to accomplish various tasks ranging from enhancing public security and safety [13] to commercial drone-delivery operations [14]. The inherent flexibility, small form factor and low-power consumption make FPGAs potential components to address the growing design challenges of UAVs, that is, the tradeoffs between performance, power, security and reliability. CNN models have been used to improve the efficiency and achieve autonomous navigation for self-driving cars, robots and drones [15, 16].

In this context, this work *contributes* by promoting a novel energy-efficient coarse-to-fine (C2F) inference model, which coarsely classifies input images into superclasses and then executes the appropriate fine model to recognise and rank the input images according to their bespoke categories. The concept of C2F, forming the hierarchical architecture and producing the results at different levels of granularity, has been adopted in various fields of study, including fashion [17], medical [18] and agriculture [19]. In this article, the underlying approach is leveraged to demonstrate the advantages gained when the system decides not to progress into the fine inference part because the result of coarse inference is not included in and aligned with the classification objective. In contrast to executing the original deep model that classifies every input data into fine-grained categories, the proposed approach offers reduced inference time and power consumption. This is attributed to the shallower model used for coarse inference. Additionally, the proposed architecture exploits the PR feature, allowing multiple fine inference models and the deeper layers of the original model, stored in external memory, to be loaded and programmed on the fly. Therefore, this adaptive architecture also allows the model to reprogram into its original structure for typical classification. In design consideration, the PR time is considered as it can significantly affect the system's performance. Thus, ZyCAP [20], a custom controller with a minimised overhead time for PR, is deployed in our proposed architecture. In addition, PR is utilised only to switch between the different models which does not affect the inference speed. Results show that our approach significantly improves the classification process when object identification of only one coarse category of interest is needed. Compared to performing the original model on every input data, this approach can reduce energy consumption and inference time by up to 27.2% and 13.2%, respectively, which can greatly benefit resource-constrained applications.

The rest of this paper is organised as follows. In Section 2, we introduce the basic concepts of the FINN automation framework and PR. Then, we provide related works and the proposed approach's advantage over them. Section 3 presents the proposed reconfigurable C2F approach, including its architecture and possible operation modes. The results of the experiments for the two operation modes are presented and discussed in Section 4. Finally, the conclusion of this article and future work are given in Section 5.

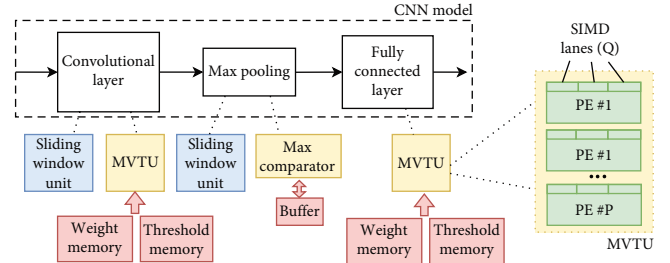


FIGURE 1: A simple convolutional neural network (CNN) mapping onto hardware architecture by FINN.

## 2. Basic Concepts and Related Works

In this section, we first discuss the importance of automation frameworks for generating FPGA-based neural networks. Then, we describe the mechanism behind the FINN automation framework and how to utilise it. After that, we give an overview of the PR feature on FPGA and introduce ZyCAP, the PR controller used in our approach. Finally, we discuss the related works and the novelty of our proposed approach.

**2.1. FINN Automation Framework for FPGA-Based Neural Networks.** Mapping CNNs onto FPGA is a nontrivial task as the behaviour of FPGA is typically described by a hardware description language (HDL) such as VHDL and Verilog. CNN computations are in general inherently parallel, which align adequately with GPU architectures. Adjusting the underlying models to a more parallel FPGA architecture may require significant algorithms and data flow changes, which can be error-prone and time consuming. Although it is possible to program an FPGA using easy-to-interpret high-level languages (e.g. C/C++), commercial tools do not offer adequate machine-learning library support. To mitigate this problem, many automation frameworks for developing FPGA-based CNNs have been introduced [21].

Intending to reduce development time, automation frameworks for mapping CNN solutions onto FPGA receive the input from popular machine learning frameworks, for example, Torch [22], Caffe [23], Keras [24] and Pytorch [25] with minimal modification. There are various automation frameworks available from commercial tools like Vitis AI [26] from Xilinx to research projects such as FINN [27, 28], hls4ml [29], fpgaConvNet [30], DNNWeaver [31] and HADDOC2 [32]. For edge developments on low-cost and resource-limited target FPGAs, the FINN automation framework, which supports implementing quantised neural networks (QNNs), that is, neural networks with low precision in weights and activations, is an ideal candidate. Furthermore, FINN is an active open-source project with a significant amount of information provided by the developers and the user community.

FINN is a research-based framework from Xilinx that allows users to map custom QNNs onto an FPGA device based on the streaming dataflow architecture, that is, each layer is implemented sequentially, shifting data through the layers. QNNs with low bit precision, especially 1-bit or binarised neural networks (BNNs), can be generated without the external memory bottleneck because the parameters can be stored on the on-chip memory. Figure 1 shows how the

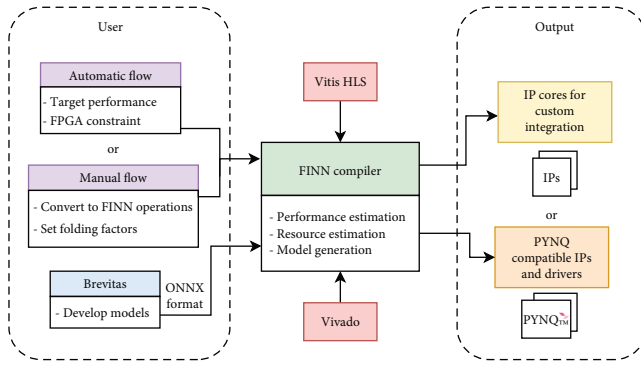


FIGURE 2: The FINN compiler tool flow.

FINN compiler maps a simple CNN containing only one convolutional, one max pooling and one fully connected layer onto a target FPGA. The convolutional layer is converted into a sliding window unit and matrix-vector-threshold unit (MVTU), while the fully connected layer comprises only the MVTU. The sliding window unit slides throughout the entire input feature map to select the group of elements that are multiplied by the weight matrix in the MVTU. The same rule applies to the hardware module of the max pooling layer, where the sliding window unit is responsible for feeding a group of elements, but this time they are passed to the maximum comparator. The MVTU contains processing elements (PEs), the main components for computation, each having several single instruction, multiple data (SIMD) lanes to provide parallelism.

Users can configure the number of PEs and SIMD lanes to determine the parallelism or the performance of the MVTU. Naturally, the higher performance requires the higher resources.

Figure 2 illustrates the overview of the FINN tool flow. First, users develop the network architecture and parameter training using Brevitas [33], a PyTorch library developed by Xilinx. Brevitas provides tools for users to build and train QNNs. Then, the trained QNNs are exported using the Open Neural Network Exchange (ONNX) format before submitting to the FINN compiler. After that, users can manually transform the trained model into the FINN-ONNX format using the FINN infrastructure or let FINN automatically convert the model. In this phase, the general operations can be optimised using FINN transformation functions. For instance, MAC operations for a 1-bit precision network can be transformed to XNOR, followed by pop count operations. Besides, the number of PEs and SIMD lanes, called *folding factors*, which determine the accelerator's parallelism can be adjusted on each layer. Therefore, this process directly affects the performance and resource usage of the generated hardware accelerator. In the mapping phase, FINN translates each layer to a corresponding Vitis HLS function call, which can later be synthesised to an intellectual property (IP) block used in Xilinx Vivado [34]. Subsequently, the generated IP blocks are interfaced with each other, forming the desired network. These IPs can be integrated seamlessly using provided drivers with PYNQ platforms or interacted in a custom design through AXI-Stream protocol [35].

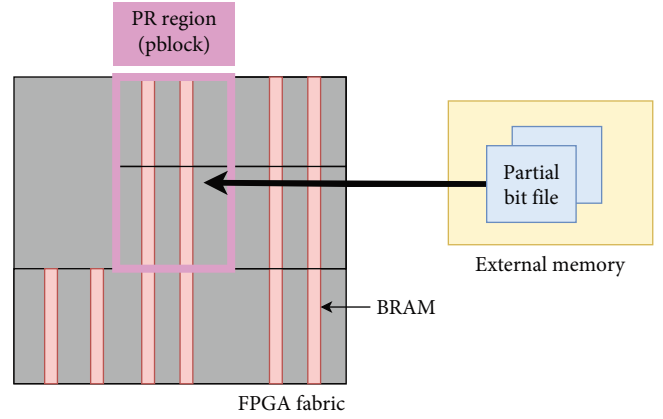


FIGURE 3: Partial reconfiguration (PR) on field programmable gate array (FPGA). The PR region is defined by the user to allocate the FPGA resources. It indicates the position of the dynamic part allowing the PR to perform later after loading from the external memory.

As the FPGAs for edge developments have limited resources, and neural network implementations are generally resource-intensive, a potential solution is using PR. Thus, instead of storing the whole deep CNN on the static FPGA, specific model parts (e.g. layers) can be loaded only when necessary, enabling different models to be deployed in resource-constrained FPGA devices.

**2.2. PR on FPGA.** PR feature enhances effective logic density through the time-multiplexing of hardware resources. Specifically, PR allows some parts of the FPGA to be reconfigurable by downloading partial bitstream files stored in the external memory while the remaining portions are not interrupted. Hence, the size of the FPGA can be optimised, which consequently obtains cost and power consumption reduction [36].

In general, if PR is enabled, users must manually define the PR region, called *pblock* as shown in Figure 3. This region allocates the resources, including the block RAMs (BRAMs), lookup tables (LUTs), flip-flops (FFs) and digital signal processors (DSPs) for the dynamic part. The size of the PR region is also directly related to the size of the partial bitstream files that are generated to be used for PR. These partial bit files contain different logic configurations to program the dynamic part to perform particular tasks. The region outside the defined pblock is the static part which is programmed once during boot time using the entire bitstream file. It accommodates permanent elements of the system (e.g. input/output ports) since they cannot be physically relocated. Essentially, the PR process initially starts with the PR controller being triggered to transfer the partial bitstream file from the memory to the configuration ports, which for Xilinx FPGA are Internal Configuration Access Port (ICAP) and Processor Configuration Access Port (PCAP). While ICAP serves as the port from the FPGA side for executing PR, PCAP enables the central processing unit (CPU) to perform the PR operation. Theoretically, PCAP's maximum throughput is 145 MB/s [37], and ICAP's maximum throughput is 400 MB/s [36] for the Xilinx ZYNQ-7000 FPGA family.

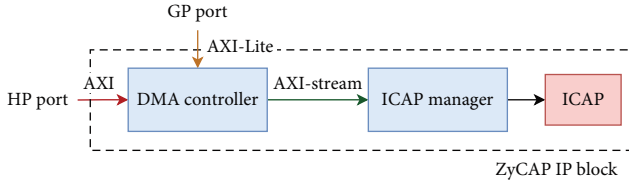


FIGURE 4: The block diagram of ZyCAP.

In our approach, the PR is performed through ZyCAP [20]. ZyCAP is a custom controller for PR via ICAP that nearly reaches the theoretical throughput of the ICAP. It achieves the PR speed of 382 MB/s, which is 95.5% of the maximum speed of ICAP. As shown in Figure 4, the ZyCAP contains a direct memory access (DMA) controller, an ICAP manager and an ICAP primitive. It allows the CPU to control the DMA by issuing the size of the partial bitstream file and the memory start address through the general purpose port (GP port). Then, the bit stream file is transferred at high speed through the high-performance port (HP port) from the external memory to the DMA controller. In the end, the ICAP manager receives the raw partial bit stream data from the DMA controller before converting it to the specific format for the ICAP.

**2.3. Related Work in FPGA-Based CNN Implementations Using PR.** Deep CNNs comprise many parameters that need high computation complexity and memory capacity. Therefore, some additional techniques are required to deploy them on FPGA efficiently. A potential method is to design FPGA CNN implementations based on compact size models. For example, the MobileNet [38] and the ShuffleNet [39] models, with their respective FPGA implementations of [40] and [41], carry few parameters, especially on convolutional layers. Another approach is to apply the model compression techniques, including model pruning [42–44], which is the process of removing some parts of the model, and quantisation [45, 46], which decreases the number of data bit widths to provide faster inference time with lower memory requirement.

BNN refers to a network that quantises the data precision, typically a 32-bit floating point, into a 1-bit representation. This form of extreme quantisation promotes the FPGA implementation for two reasons. First, it reduces the memory requirement, allowing the parameters to be cached on the on-chip memory. Hence, no external memory access is needed, thus improving the system's energy efficiency. Second, the MAC operation of convolutional layers can be replaced with XNOR and pop-count operation, which significantly simplifies the complexity of the FPGA implementation [27, 47].

FPGA-based implementation of CNNs using PR has been explored for various purposes, such as saving energy or improving performance. In [48], an FPGA-based approach deciding an early exit from the deep CNN model named adaptive and hierarchical convolutional neural network (AH-CNN) is presented. This work uses the PR to load the CNN's deeper layers when the inference model's confidence value is below a specified threshold. As a result, the model decreases the inference time when the confidence level

of the shallow model is acceptable and provides the opportunity to increase the accuracy when the confidence level is too low by letting the images go through the deep part of the model. The work in [49] divides the CNNs into chunks of layers. To minimise the resource utilisation, one chunk is programmed at a time. Once the first chunk is executed, the next chunk in sequence is downloaded to the FPGA using PR. This process is repeatedly performed to progress to the last chunk of the network.

Authors in [50] propose a reconfigurable convolution PE for three different bit-width precision designs of 8, 16 and 32 bits. Instead of downloading the full bit stream file to switch among the matrix accelerator designs, they can be specifically reconfigured using PR with a smaller size of partial bit file. In [51], the use of PR is demonstrated for boosting the flexibility of CNN accelerators. The fully connected layers and the first convolutional layer of the proposed architecture are defined to be reconfigurable partitions. Therefore, they can be changed to the dedicated layers of the models trained for a specific classification task. Three use cases are demonstrated in this paper. First, the first convolutional layer can be remodelled from classifying the Modified National Institute of Standards and Technology (MNIST) dataset [52] to the Canadian Institute for Advanced Research (CIFAR)-10 dataset. Second, the fully connected layers, which are independently trained with the MNIST and Extended MNIST (EMNIST) datasets [53], are switchable for the letter and digit classification, respectively. Finally, these fully connected layers can also be reconfigured to spiking neural network (SNN) to improve the accuracy of the digit classification. In [54], the PR is applied to reduce the number of bit representations of the inference model when the battery level of the system decreases. This approach can prolong the duration of system use with the same battery capacity.

PR implementations with lower throughput can add some overhead, which can lead to high response times—gold criteria for resource-constrained devices. Table 1 shows that most reviewed works rely on low-speed ICAP or PCAP controllers to perform the PR, which adversely affects performance and energy efficiency.

Our work differs from the existing ones in two major areas. While the PR throughput is overlooked in every design, we are the first to implement the cutting-edge ZyCAP for PR, which attains the highest throughput compared to the other implementations. This implementation would significantly improve performance and energy consumption, vital edge devices' requirements. The second major advantage of our work is the novel C2F approach, which differs from how a typical classification is carried out, with less resource demand. The inference model classifies an input image into a coarse category first and then provides the fine category of the given image. In this manner, once the coarse inference results are received, the system can decide whether to continue propagating through the fine stage. The energy efficiency obtained through this method is significant (along with slight accuracy improvement) for applications where a superclass of objects among many should be classified, and others can be safely ignored. When the classification objective is changed, our



TABLE 1: FPGA-based CNN accelerators exploiting preconfiguration.

| Authors                        | FPGA device  | PR port/controller | PR throughput (MB/s) |
|--------------------------------|--------------|--------------------|----------------------|
| Youssef et al. [54]            | ZYNQ XC7Z020 | ICAP               | 10                   |
| Cain et al. [50]               | ZYNQ XC7Z020 | JTAG               | 66                   |
| Irmak et al. [51]              | ZYNQ XCZU9EG | PCAP               | 145                  |
| Seyoum et al. [49]             | ZYNQ XC7Z020 | N/A                | N/A                  |
| Farhadi, Ghasemi and Yang [48] | ZYNQ XC7Z020 | N/A                | 60                   |
| Our work                       | ZYNQ XC7Z020 | ZyCAP              | 382                  |

Abbreviations: CNN, convolutional neural network; FPGA, field programmable gate array; ICAP, Internal Configuration Access Port; JTAG, Joint Test Action Group; N/A, not applicable; PCAP, Processor Configuration Access Port; PR, partial reconfiguration.

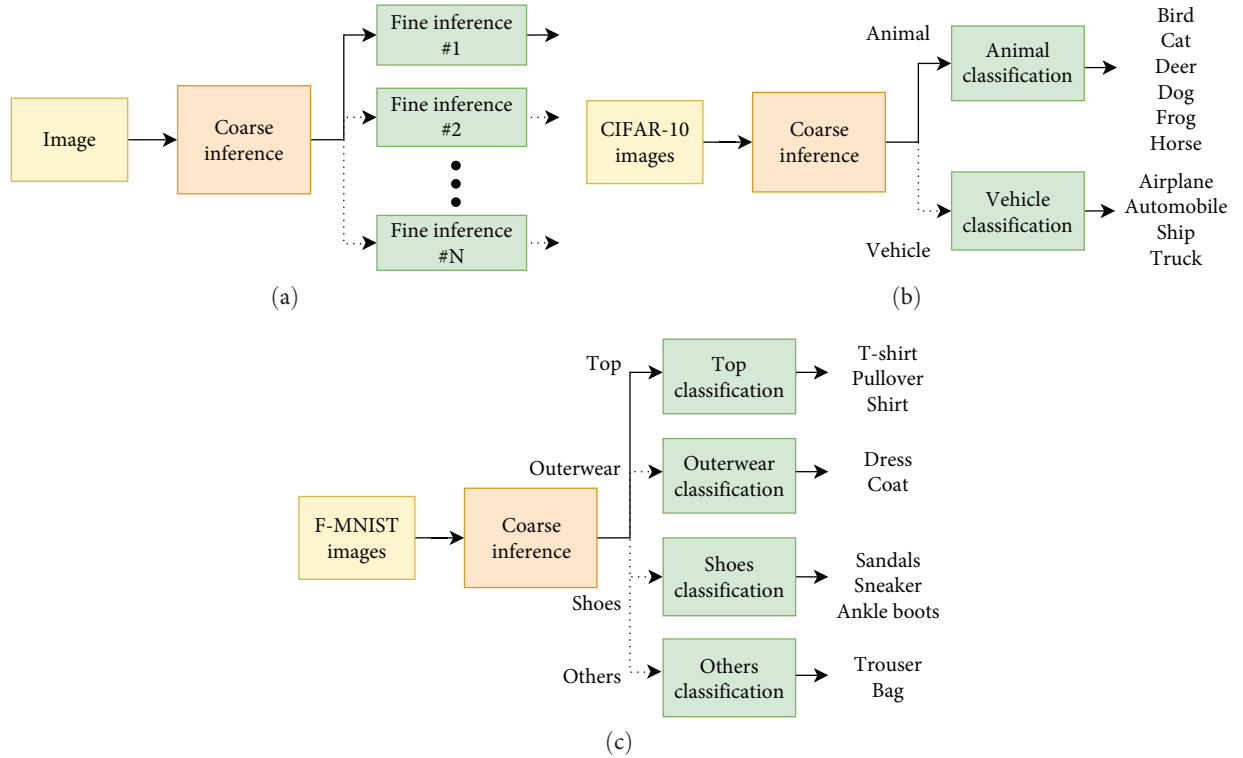


FIGURE 5: Diagram of the coarse-to-fine (C2F) inference model: (a) a generic model, (b) applied to CIFAR-10 dataset and (c) applied to F-MNIST.

enhanced PR technique is used to dynamically switch between various fine inference models stored in external memory. This approach contrasts with statically programming multiple models onto the FPGA fabric, a process demanding substantial FPGA resources. For applications where every object should be classified, PR can be utilised to reprogram the deeper layers to restore the original model.

### 3. Proposed Dynamic Reconfigurable C2F Approach

This section presents the proposed reconfigurable C2F approach, which enables the development of energy-efficient FPGA-based CNN inference models. First, it describes the design process of the proposed approach using the CIFAR-10 dataset [55] and Fashion-MNIST (F-MNIST) [56] as examples. This section also gives an overview of how the

available tools can be used to develop FPGA-based CNN implementations. Then, the structure of developed inference models is presented. Finally, an example of hardware implementation and possible operation modes are discussed.

**3.1. FPGA-Based C2F Inference Approach.** The proposed C2F approach is inspired by the hierarchical deep convolutional neural network (HD-CNN) [57] and decomposes an image classification task into two stages. First, the items are categorised at a broader level by a coarse category CNN classifier. Then, for a local classification, the items are routed downstream into fine category classifiers, as illustrated in Figure 5a. This method provides flexibility in performing the second stage, that is, fine inferencing. Depending on the application and the received item for processing, once the coarse inference is applied, the system can decide to directly use the complementing default fine inference implementation currently on

FPGA for further classification, or to exit processing immediately after coarse inference stage.

Naturally, constructing coarse categories based on the shared image features (e.g. images containing a head, body and legs are grouped as an animal category) would increase the coarse model's accuracy since the model can easily recognise distinct patterns among the different feature-grouped categories. On the other hand, ad hoc grouping coarse categories without considering shared features might reduce the coarse model's accuracy because the image features of different coarse categories might be similar and difficult to differentiate. Hence, designers should consider the trade-off between accuracy performance and similarity in image features during the coarse-grained categories' construction. Another factor to consider is the activation frequency of the fine model. Essentially, fewer executions of the fine model lead to higher overall inference efficiency because executing only the coarse model requires less time and power consumption compared to executing both the coarse and fine models. This implies that to fully exploit the advantages of the approach, the target dataset should contain only a small percentage of images that need to be categorised into fine-grained labels.

With the above considerations, the C2F concept is applied to two public datasets for evaluation. The first example is for classifying the CIFAR-10 dataset. An intuitive approach based on the shared image features is to have two categories of 'vehicle' and 'animal' for the coarse classification as shown in Figure 5b. The fine inference model for the 'vehicle' superclass should contain four subclasses, 'airplane', 'automobile', 'ship' and 'truck'. There are also six subclasses for the 'animal' superclass, which are 'bird', 'cat', 'deer', 'dog', 'frog' and 'horse'. Another dataset example is F-MNIST, containing the images of wearable objects. We assigned the coarse-grained categories for the F-MNIST as four categories illustrated in Figure 5c. The first fine inference model is 'top', which contains the 'T-shirt', 'pullover' and 'shirt' subclass. The second fine inference model is named 'outerwear', which consists of 'dress' and 'coat'. The third fine inference model is the 'shoes', which accommodates the 'sandals', 'sneaker' and 'ankle boots' labels. Finally, the last fine inference model is 'others', which are 'trouser' and 'bag'. Even though the objects in the 'others' group have no obvious shared features, their features are noticeably different compared to the other coarse categories.

Figure 6 illustrates the overview of utilising available tools to implement the proposed model on an FPGA. At first, CNN models are built and exported in ONNX format using the Brevitas Python library. In this stage, the constructed models are trained and evaluated, including the original model and the C2F model, based on PyTorch framework's facilities. The original model is constructed using a typical machine learning development flow, that is, model training and evaluation with the whole dataset. In the C2F model, the coarse model and the fine model, which are defined based on the original model (further discussion in Section 3.2), are connected together for training and evaluation with the particular coarse category dataset. For example, in Figure 6a, the coarse model is joined with the fine model

#1. Hence, training and evaluation are performed with the dataset corresponding to the fine model #1. The underlying approach is generic, that is, it is applicable to any dataset. Then, the exported model is translated into FINN's operations, and Xilinx IP cores are produced as outputs using auto-generation flow from FINN infrastructure. To ensure that generated CNN accelerators from FINN fit with the target FPGA, the FINN analysing tool can be used to estimate the resource utilisation rather than employing Xilinx Vivado to generate the real synthesised design and the resource utilisation report, which is a time-consuming process. If the required resources exceed the available ones on the FPGA, the accelerator's target performance given to FINN can be reduced to decrease the resource requirement. These steps are repeated for generating the coarse and fine inference model IP cores. Additionally, to allow the system to reprogram back to the original configuration, the deeper layers of the original model is also generated. The generated models are then integrated with other components in Xilinx Vivado, as shown in Figure 6(b). One of the fine models is synthesised to allocate resources for the dynamic part or *pblock*. With enabling PR, the other fine models and the original model's deeper layers, stored in the external memory, can be switched on to the allocated dynamic region at runtime. The rest of the components, including the coarse model and ZyCAP IP core, are defined as the static part and can be programmed only once at the boot time.

### 3.2. C2F Inference CNN Model Architecture

**3.2.1. The Original Model.** In a typical architecture of a CNN design, an input, for example, an image, is passed through a number of layers including convolutional, max pooling and fully connected, as can be observed in Figure 7, showing the CNN model named CNV which is one of the pretrained models provided by FINN for the CIFAR-10 classification. Therefore, the model accepts images with the size of  $32 \times 32 \times 3$ , which is three channels (red, green and blue (RGB) colours) of 32 pixels width and 32 pixels height, and outputs numbers in the range of 0–9 which correspond to the category of CIFAR-10 images, for instance 'airplane' as 0, 'automobile' as 1 and 'bird' as 2. The CNV model consists of six convolutional layers (CONV1–CONV6) and 2 max pooling layers followed by three consecutive fully connected layers (FC1–FC3) of which the last layer has 10 outputs, arranged similar to the structure of VGG-16 model [58]. On every convolutional layer and fully connected layer, batch normalisation is used to improve the training speed and stability [59]. The kernel size of each convolutional layer is  $3 \times 3$  with no padding. The class with the highest possibility from these outputs is selected as the category prediction of the model. While the CNV model was originally designed for the CIFAR-10 dataset, it can undergo some adjustments to adapt to other datasets. The F-MNIST dataset is used to demonstrate the underlying flexibility of our approach. First, since the size of the F-MNIST is  $28 \times 28$  pixels, these input images must be resized to  $32 \times 32$  pixels before passing into the CNV model. Second, due to the images in the F-MNIST dataset containing only one channel grayscale color, the

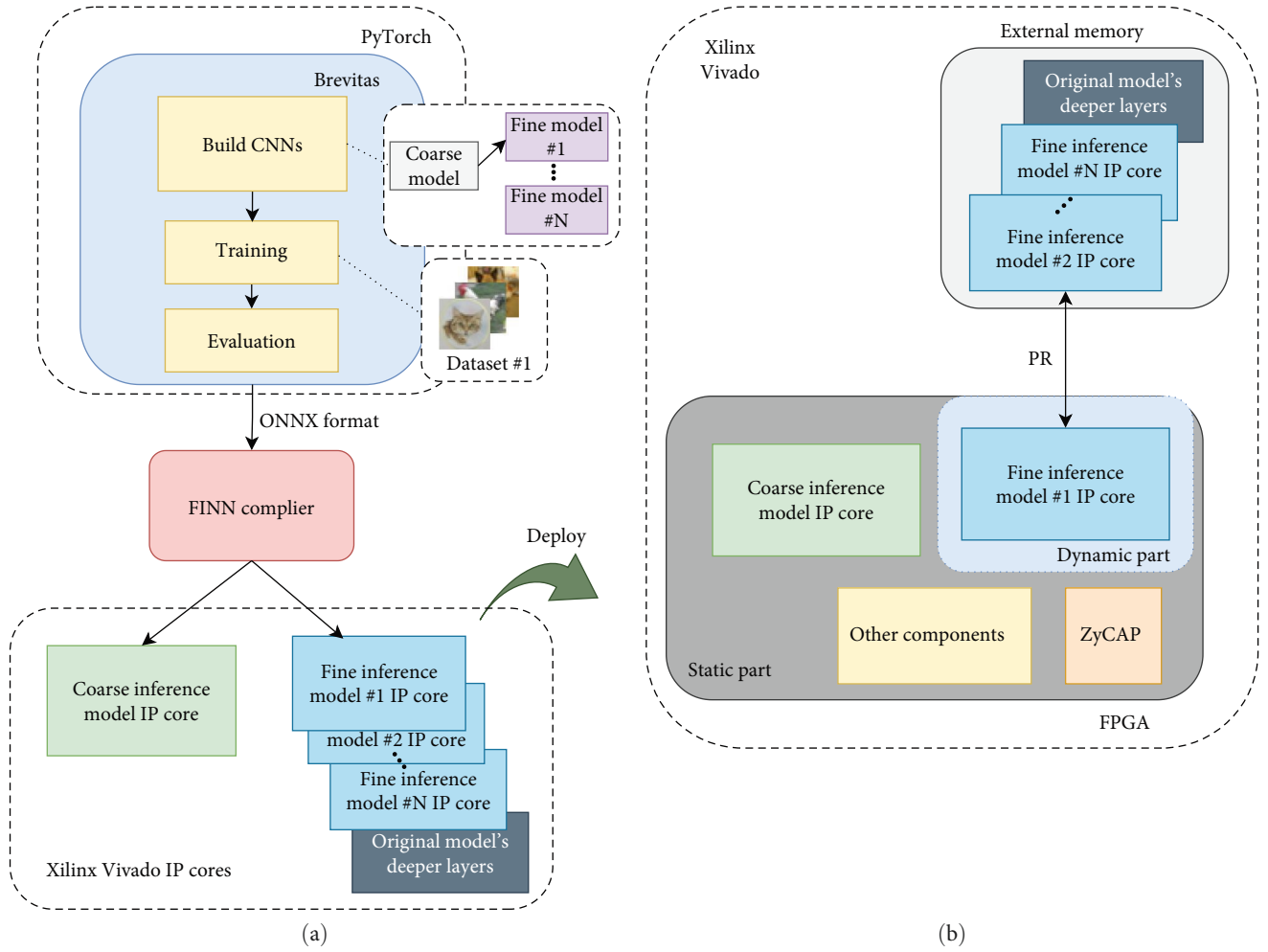


FIGURE 6: Generic workflow for coarse-to-fine (C2F) inference field programmable gate array (FPGA) implementation: (a) develop and generate FPGA-based convolutional neural networks (CNNs) from FINN and Brevitas, and (b) integrate generated CNN implementation on FPGA using partial reconfiguration (PR) for switching fine inference models and deeper layers of the original model.

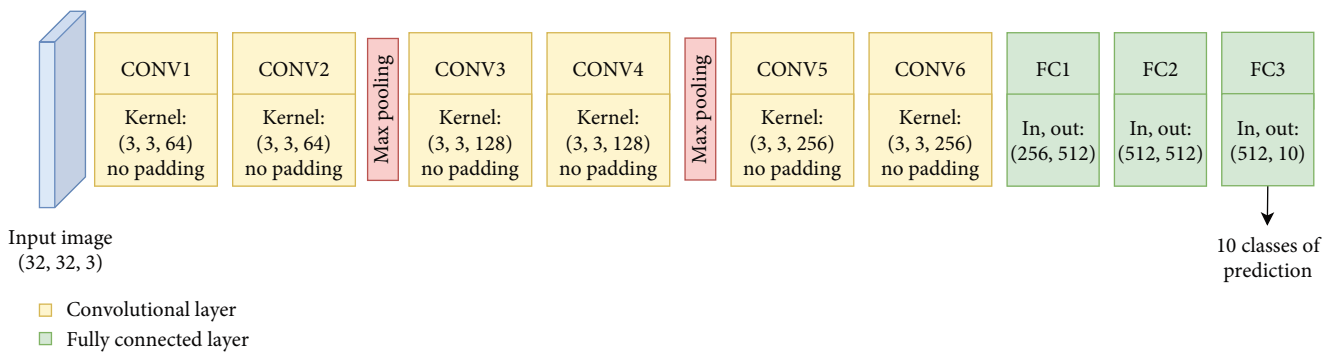


FIGURE 7: The model architecture of CNV as the pretrained model provided by FINN framework.

first convolutional layer is modified to accept the one-channel images. Finally, the whole model's parameters are initialised, trained and evaluated with the F-MNIST dataset, which is further discussed in Section 4.1.1.

**3.2.2. The C2F Model.** Our approach breaks the dataset into superclasses and classes, and correspondingly the architecture

down to two sub-models, coarse and fine models as shown in Figure 8 with *config1* and *config2*. Alternatively, by connecting the shared layers of the coarse model with the deeper layers of the original models (*config0*), our model can be constructed back into the original model for traditional classification. For the CNV model, the coarse inference model comprises the shared layers, which are four convolutional

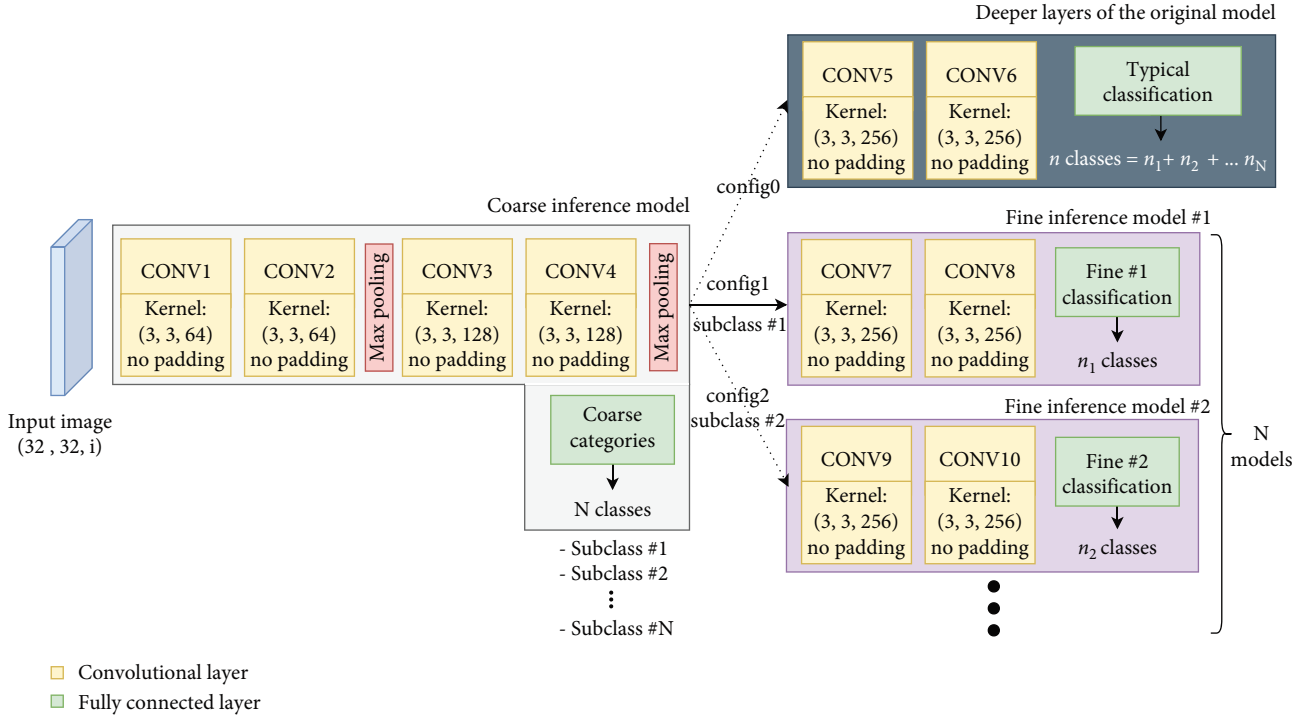


FIGURE 8: The model architecture of our coarse-to-fine (C2F) approach applied with  $N$  coarse-grained classes. The model can also be formed into the original model by connecting the shared layers with the deeper layers of the original model for  $n$ -class classification where  $n = n_1 + n_2 + \dots + n_N$ .

(CONV1–CONV4) and two max-pooling, and three new fully connected layers. The shared layers can be defined with more or fewer layers depending on the coarse model accuracy and execution speed requirement. Generally, the higher the number of layers, the higher the coarse-grained accuracy. However, constructing the shared layers with too many layers can decrease the coarse model's inference speed and harm the fine model's accuracy because there are just a few parameters of the remaining layers to be trained for the fine-grained classification. Because the shared layers are taken from the original model with identical parameters to the original model (the model training will be discussed in Section 4.1.1), the later part of the model can be the fine inference model trained with the specific dataset or the deeper layers of the original model for the fine-grained inference or typical classification, respectively. For the CNV model, the later layers include two convolutional and three fully connected layers.

For the CIFAR-10 dataset, the coarse inference model is responsible for grouping the input images into two super-classes, which are 0 for 'animal' and 1 for 'vehicle', while the images are grouped into four superclasses, which are 0 for 'top', 1 for 'outerwear', 2 for 'shoes' and 3 for 'others' for the F-MNIST dataset. In correlation with the output of the coarse model, there are two fine models for CIFAR-10 dataset and four fine models for F-MNIST dataset. All fine models have a similar structure except for the output of the last fully connected layer depending on the number of the fine-grained classes. For example, for CIFAR-10 dataset, the last fully connected layer of the 'animal' category has six outputs

and the 'vehicle' category has four outputs. With the given model structure, the original model parameters are reduced from  $\sim 1.55$  to  $\sim 0.69$  M for the coarse model, corresponding to a reduction of 55%.

Hence, the energy consumption and inference time are expected to decrease if the coarse model is executed instead of the original model.

**3.2.3. Hardware Implementation.** Figure 9 shows the developed hardware architecture used to validate the proposed reconfigurable C2F approach. The resulting implementation was deployed in a Zedboard board, including a Xilinx Zynq SoC consisting of a processing system (PS) and a programmable logic (PL) area. While developed custom modules (i.e. selector, data decoupler and duplicator) were generated with Vitis HLS, FINN was used to produce related CNN modules (e.g. convolutional layers). To achieve maximum performance, the ZyCAP system runs at 100 MHz, while the other components run at only 71.42 MHz due to the constraint from the critical path delay.

To exploit the massively parallel computation capability of the FPGA, the process of inference is carried out on the PL side. Conversely, the PS handles tasks that require sequential processing and low computational demands. PS issues the DMA to move the image data from the external memory (DDR3) on Zedboard into the coarse convolutional layers module and also issues ZyCAP through the HP1 port to fetch the partial bitstream file from the DDR3 and perform the PR, which reprograms the dynamic part with either one of the fine models or the deeper layers of the original model. In



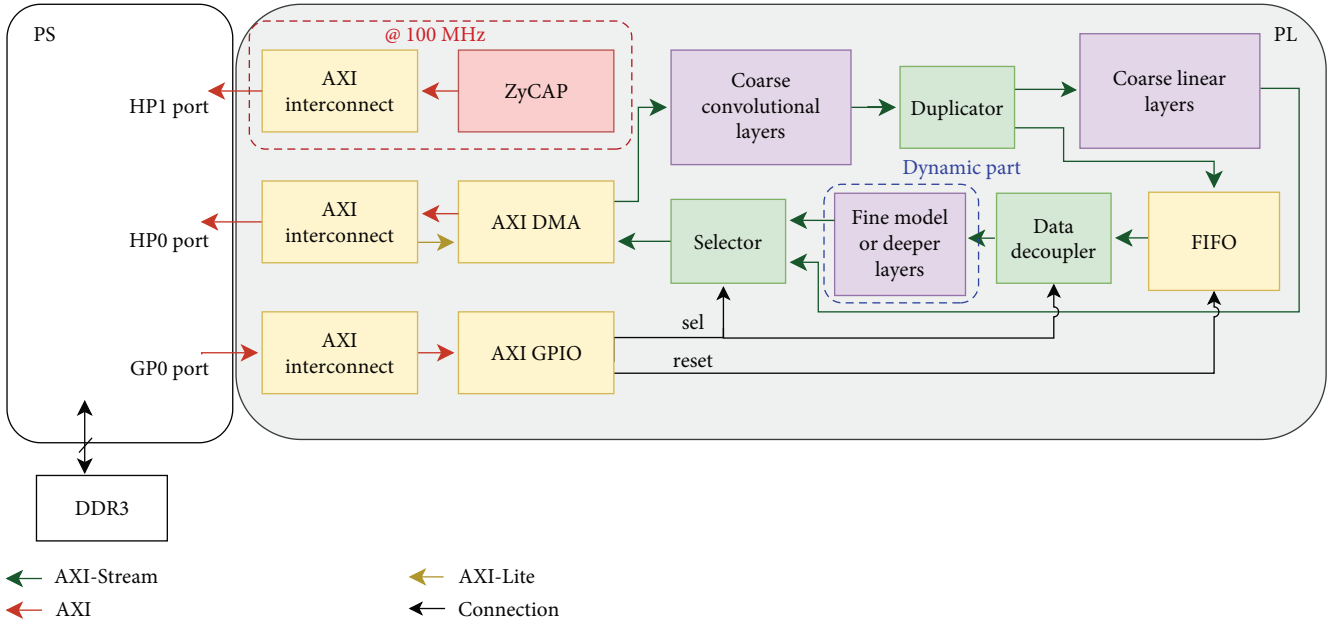


FIGURE 9: The overall hardware architecture of dynamic reconfigurable design of the coarse-to-fine (C2F) inference implementation. Each arrow indicates the direction of data flow, for example, from master to slave for AXI-based protocol and from the output of the component to the input of another component for normal connection.

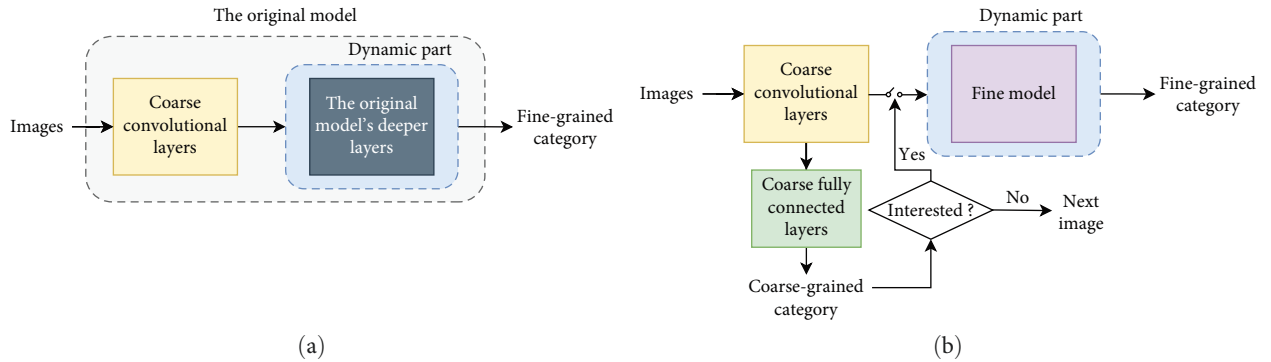


FIGURE 10: Diagram of operation mode of the proposed architecture: (a) operation mode 1 and (b) operation mode 2.

addition, after receiving the result from the coarse inference, PS decides whether to advance into the fine inference model through the AXI GPIO module.

The inference process is initiated from an HP0 port interfaced with AXI DMA by the CPU. The sample inputs are directed to the coarse convolutional layers module outputting as the feature map which are then passed through to two separate paths; one going through the coarse linear layers module and the other to be kept in the first in first out (FIFO), which is mainly built from BRAM. The superclass of a sample image is generated from the coarse linear layers. After that, the CPU deasserts the *sel* signal through the GP0 port to control the selector to connect the AXI DMA with the coarse linear layers as well as to command the data decoupler module for disconnecting the data flow between the dynamic part, which can be either the fine model or the deeper layers of the original model, and the FIFO. Thus, the CPU can receive the superclass result through the AXI DMA

module while buffering the output of the coarse linear layers on the FIFO, which may be passed to the dynamic part in the future. This *sel* signal from CPU through AXI GPIO is also used to disable and enable the clock enable port to optimise the power consumption of the dynamic part when it is not operating. Later on, the CPU decides if the inference needs to be progressed. If yes, the *sel* signal is asserted letting the FIFO pass the stored feature map to the dynamic part and generate the fine inference result and pass it to the DDR3 via the DMA. The result can then be read by the CPU. Conversely, if the coarse result does not fall in the objective category, the CPU issues the reset signal to the FIFO to flush the stored data and start a new inference for the next image.

Two operation modes based on the proposed architecture using PR to reprogram the dynamic part of FPGA, as illustrated in Figure 10, are presented in the following sections. The first mode is to classify all images into the fine category. In the second mode of operation, certain input

```

1. Require: Input Image ( $X_i$ ), Interested Coarse category ( $IC$ )
   while  $X_i$  is Valid do
     /*  $C$  is the superclasses from the Coarse inference */
     /*  $FM$  is the output Feature Map of
        the Coarse convolutional layers */
2.    $C, FM = \text{Coarse-Inference}(X_i)$ 
3.   if  $C == IC$  then
4.      $F = \text{Fine-Inference}(FM)$  return  $F$ 
5.   else
6.     return // No further action required, restart the loop

```

ALGORITHM 1: Interested item classification with C2F model.

items are relevant to our application while the others can be ignored. For example, in its simplest form, all the classes of the dataset are grouped into two superclasses, one containing the classes of interest and the other the unwanted classes. This superclass inferencing is carried out in the coarse model part of the architecture. The outcome of this model would specify whether the current input sample falls within the superclass of interest. If yes, this will go through the default fine model which is already on FPGA to identify the exact class of the input sample. If not, then the coarse model is the only architecture applied to the input sample and process breaks out there without carrying out any fine processing. This termination of the inference process after the course stage for unwanted objects significantly improves energy consumption of the application due to less computation cost compared to executing the original full model.

**3.3. Operation Mode 1: Typical Classification.** In this mode, the system directly classifies input images into fine-grained categories and ignores the coarse-grained class result. Since the convolutional layers of the coarse model comprise the parameters of the first part of the original model, the original model can be rebuilt using PR to reprogram the FPGA's dynamic part with the deeper layers of the original model as shown in Figure 10a. Therefore, when the system needs to classify every object into the fine-grained categories, the overall inference time can be reduced compared to the C2F model, which requires additional execution of the coarse fully connected layers.

**3.4. Operation Mode 2: Interested Item Classification.** To activate the operation mode 2, the fine model is programmed in the dynamic part of the FPGA using PR as shown in Figure 10b. In some scenarios, the objective of the image classification is to finely identify only one coarse category and the other coarse categories can be safely discarded. For example, the purpose of an image classification could be to recognise the type of animals in the CIFAR-10 dataset. In this example, there are images for which the fine classification is not necessary, which are vehicle images. Hence, these vehicle images just need to be recognised as 'vehicle' allowing us to safely drop them without further processing. The procedure of this approach is summarised in Algorithm 1. Using

this approach with this particular objective, the expected performance in terms of power consumption and latency should be improved because the coarse inference model is shallower and requires a lower computation cost compared to the original model. The PR can still be used to switch between the fine models enabling the flexibility for the application while the system is running, for example, when the user shifts the interest from one superclass to another at run-time without worsening the model execution time.

## 4. Experimental Setup

**4.1. CNN Model Preparation.** We adopt the pretrained FINN CNV model with 1-bit weights and 2-bit activations named CNV1W2A. The primary reason for this selection is that resource requirements for CNV1W2A model is attainable for implementation on the resource-constrained Zedboard. Moreover, this implementation does not require DSP as the data precision for the CNV1W2A is low. To build the Coarse inference model for CIFAR-10 classification, the first four convolutional layers were taken from the pretrained CNV1W2A and their parameters were set as fixed. With the fixed parameters, these convolutional layers can be reused among coarse and fine models without biasing on the particular dataset during the model training. These convolutional layers were then connected with 3 fully connected layers where the last layer's output was 1 for 2 superclasses (0 and 1). Then, the coarse model was trained with the CIFAR-10 dataset which was relabelled to 'vehicle' and 'animal'. For the fine classification, the convolutional layers of the coarse model were reused by connecting the coarse model with two convolutional layers and three fully connected layers as previously shown in Figure 8. The last fully connected layer's output was 6 for animal classification and 4 for vehicle classification. Finally, every fine model was trained with the labels corresponding to CIFAR-10 dataset excluding the other category. To be specific, the animal inference model was trained with 30,000 animal images, and the vehicle inference model was trained with 20,000 vehicle images from the CIFAR-10 dataset.

Building the coarse and fine models for the F-MNIST dataset was similar to the process above with additional modifications. First, due to the F-MNIST images containing greyscale data, the first layer of the convolutional layers of the CNV1W2A was modified to accept 1-channel input instead of 3-channel input. Then, the modified CNV1W2A's parameters must be reinitialised and trained with the F-MNIST dataset to build the model for typical classification. Subsequently, the steps for constructing the model can be repeated, following a process similar to that employed for the CIFAR-10 dataset. The output of the coarse model's last fully connected layer is 4 for four superclasses ('top', 'outerwear', 'shoes' and 'others').

For both F-MNIST and CIFAR-10 datasets, the training process for the coarse and fine models were performed using PyTorch and the training images were preprocessed with the standard augmentation including random cropping and flipping. The F-MNSIT dataset was also resized to  $32 \times 32$  pixels in this data preprocessing. The learning rate was 0.02 at the

TABLE 2: Accuracy comparison between the original and our model for CIFAR-10 dataset.

| Model               | Original model | Our model     |
|---------------------|----------------|---------------|
| All fine categories | <b>87.76%</b>  | —             |
| Coarse category     | —              | <b>97.15%</b> |
| Animal category     | 84.58%         | <b>85.02%</b> |
| Vehicle category    | 92.53%         | <b>93.55%</b> |

Note: The bold indicates the best performance.

TABLE 3: Accuracy comparison between the original and our model for F-MNIST dataset.

| Model               | Original model | Our model     |
|---------------------|----------------|---------------|
| All fine categories | <b>93.22%</b>  | —             |
| Coarse category     | —              | <b>96.16%</b> |
| Top category        | 85.30%         | <b>87.97%</b> |
| Outerwear category  | 92.75%         | <b>97.70%</b> |
| Shoes category      | 97.43%         | <b>97.80%</b> |
| Others category     | 99.25%         | <b>99.95%</b> |

Note: The bold indicates the best performance.

beginning of the training, and it was gradually reduced by a factor of 0.5 in every 40 epochs. The model training was stopped after reaching 500 epochs, and the model with the highest accuracy was chosen. Finally, these models were passed to FINN to generate the FPGA accelerators in the automatic flow mode.

**4.2. CNN Model Accuracy.** The accuracy of the models is compared in Tables 2 and 3 for CIFAR-10 and F-MNIST datasets, respectively. Undoubtedly, most of the fine model trained for the specific classification, either animal or vehicle, has better accuracy than the original model, which is trained for classifying all the fine-grained classes.

## 5. Experimental Results

In this section, we first give the details regarding the training and evaluation of the CNN models. Next, the resource utilisation of the FPGA implementation is given. After that, our hardware architecture is evaluated with the CIFAR-10 and F-MNIST dataset based on operation mode 1 and operation mode 2.

Our fine inference accuracy for animal and vehicle categories is 0.44% and 1.02% higher than the original model, respectively, while the Fine inference accuracy for top, outerwear, shoes and others is 2.67%, 4.95%, 0.37% and 0.70% higher, respectively.

Additionally, the trained coarse inference model achieves the accuracy of 97.15% and 96.16% for CIFAR-10 and F-MNIST model, respectively. This proves that this method not only offers significant advantages such as flexibility and reduced power consumption but also improves the inference accuracy if the input images belongs to only one coarse category.

**5.1. Hardware Setup and Resource Utilisation.** The hardware for our proof-of-concept is a low-cost FPGA development

board called Zedboard, which contains Zynq7000 XCZ020. The proposed architecture with the PR region is synthesised and implemented using Xilinx Vivado 2022.2, and the custom modules, which are selector, duplicator and data decoupler, are described in C++ programming language and generated using Xilinx Vitis HLS 2022.2. The FPGA floorplaning along with the defined PR region (*pblock*) and its implementation is shown in Figure 11.

In Xilinx Vivado implementation, the size of the partial bit stream file for PR is 2 MB, while the full bit stream file size is 4 MB. In the dynamic or PR region of the FPGA, there are the animal and the vehicle inference parts for the CIFAR-10 dataset, as well as the top, outerwear, shoes and the others inference parts for the F-MNIST dataset. In addition, the deeper layers of the original are also implemented in this particular dynamic region. The rest of the components, including the coarse model and ZyCAP, are in the static part. The resource utilisation of each implementation can be seen in Table 4. Overall, as anticipated, the resource utilisations are quite similar across each fine inference part for both CIFAR-10 and F-MNIST, given the similar structure of the fine inference models. Notably, the use of 1-bit width for weights and 2-bit width for activations eliminates the need for dedicated DSP units. This helps with minimising power consumption. It is worth mentioning that in our approach the number of BRAMs required surpasses 100% of the available resources, meaning that the larger FPGA would be needed if the design was developed without PR.

**5.2. Evaluation on Operation Modes 1 and 2.** In this section, we evaluate the operation modes 1 and 2 of the proposed hardware architecture previously described in Section 3.4 and Section 3.5. The execution times for different tasks are measured and reported in Figure 12. For these measurements, the timer counter triggered from PS is used. Since the fine model and the deeper layers of the original model are using the same structure, their execution time is identical, which is 1.1 ms. To perform a typical classification in operation mode 1, the model takes 3.8 ms, while the coarse model takes only 3.3 ms, which is 13% faster.

The C2F model, categorising into superclasses and then into subclasses, requires 4.4 ms of execution time. Lastly, the PR time is 5.2 ms, which is still taking longer than any inference processes. Fortunately, rather than performing PR during the inference process, which could degrade overall inference speed, our approach utilises PR to alter the deeper layers of the model only when switching between model configurations in the rare event that the classification objective changes. For instance, in a surveillance robot, the classification focus may shift from one target (e.g. car) to another (e.g. a person) due to task requirements to change targets of interest. Furthermore, as discussed in Section 2.3, compared to other reconfiguration controllers, ZyCAP is the fastest, hence the energy consumption due to the PR process is efficiently minimised.

Another experimental evaluation is based on operation mode 2 from Section 3.5 where some categories of interest need to be finely classified. This mode provides timing advantages which translates into improved energy efficiency as given in Table 5.

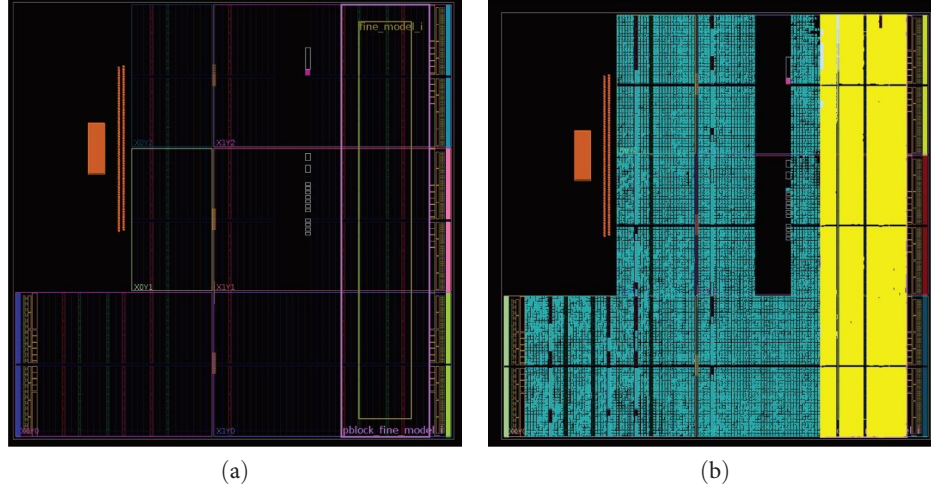


FIGURE 11: (a) The field programmable gate array (FPGA) floor planning for defining partial reconfigurable part in the pink box (*pblock*) and (b) the layout of the FPGA implementation where the reconfigurable part is highlighted in yellow and static part in blue.

TABLE 4: FPGA resource utilisation.

| Implementation |                          | BRAMs<br>(36 Kb) | LUTs   | FFs    | DSPs |
|----------------|--------------------------|------------------|--------|--------|------|
| CIFAR-10       | Static part              | 57.5             | 29,276 | 38,412 | 0    |
|                | Original model's layers  | 52               | 9679   | 14,915 | 0    |
|                | Animal inference part    | 51.5             | 9608   | 14,908 | 0    |
|                | Vehicle inference part   | 51.5             | 9608   | 14,903 | 0    |
| F-MNIST        | Static part              | 53.5             | 27,389 | 37,426 | 0    |
|                | Original model's layers  | 52               | 9681   | 14,919 | 0    |
|                | Top inference part       | 51.5             | 9590   | 14,902 | 0    |
|                | Outerwear inference part | 51.5             | 9399   | 14,900 | 0    |
|                | Shoe inference part      | 52               | 9392   | 14,900 | 0    |
|                | Other inference parts    | 51.5             | 9400   | 14,901 | 0    |

Abbreviations: BRAMs, block RAMs; DSPs, digital signal processors; FFs, flip-flops; LUTs, lookup tables.

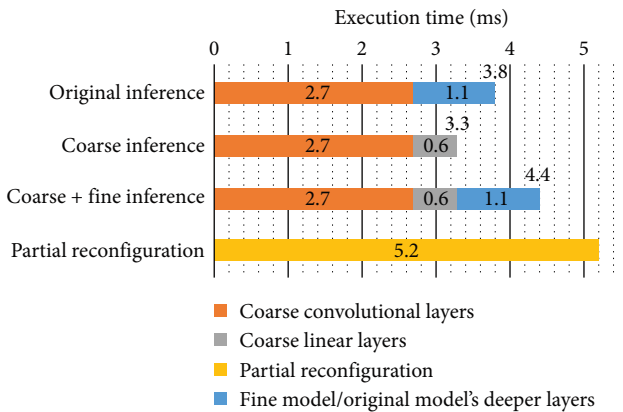


FIGURE 12: Execution time on each task.

Table 5 results, estimated using Xilinx Vivado toolset, show the advantage of the coarse inference model over the original model in terms of energy usage. The original model design consumes 0.322 W for CIFAR-10 and 0.299 W for the F-MNIST dataset. To optimise the power consumption of

our hardware architecture, which consists of the coarse and fine inference accelerators, the clock of the fine model is disabled via the clock enable port during the coarse inference execution. For instance, the power consumption of the coarse model inference for the CIFAR-10 is only 0.269 W (16.4% less than the original model), comprising 0.249 W from the coarse inference accelerator itself and 0.020 W from the inactive fine inference accelerator. The use of clock enable also applied to the model for the F-MNIST dataset and obtained the coarse model with a power consumption of 0.253 W (15.4% reduction compared to the original model). Furthermore, because the processing time of the coarse inference is less than the original inference, executing the coarse inference can save up to 27.2% and 26.7% in terms of energy consumption for CIFAR-10 and F-MNIST datasets, respectively.

In operation mode 2, since the fine model is not active unless the result from the coarse model indicates that the image is in the target category, the energy consumption is varied depending on the input image. For instance, if 20% of the dataset requires fine-grained inference, while the remaining 80% does not, the power consumption per inference



TABLE 5: Power and energy consumption per inference for the original and coarse model.

|          | Model          | Power (W)    | Processing time (ms) | Energy (mJ)  |
|----------|----------------|--------------|----------------------|--------------|
| CIFAR-10 | Original model | 0.322        | 3.8                  | 1.22         |
|          | Coarse model   | <b>0.269</b> | <b>3.3</b>           | <b>0.888</b> |
| F-MNIST  | Original model | 0.299        | 3.8                  | 1.14         |
|          | Coarse model   | <b>0.253</b> | <b>3.3</b>           | <b>0.835</b> |

Note: The bold indicates the best performance; that is, the lower the number, the better, in terms of power consumption and processing time.

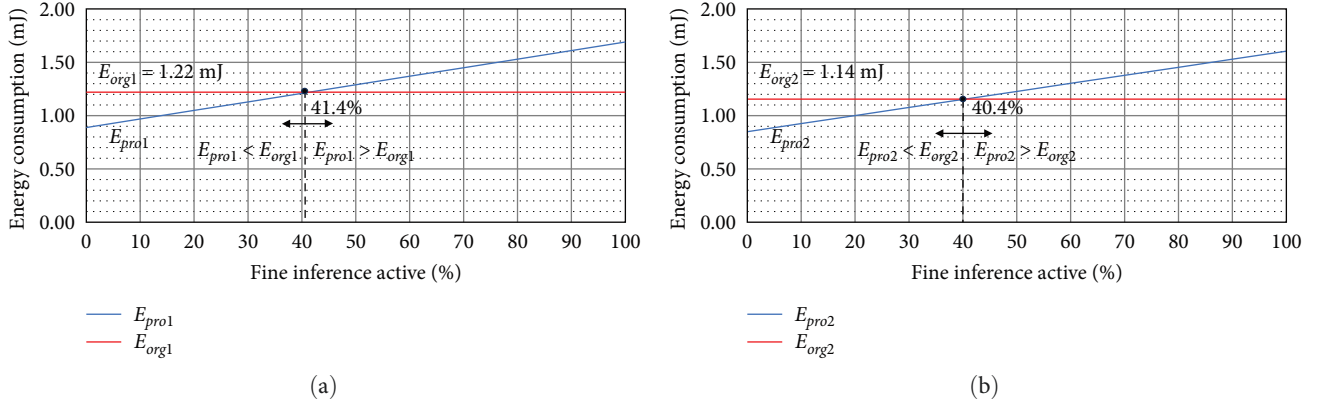


FIGURE 13: Energy consumption of the proposed method, considering the percentage of fine inference model's active time, compared with the original model's energy consumption. The energy consumption of the proposed approach ( $E_{pro1}$ : CIFAR-10;  $E_{pro2}$ : F-MNIST) is lower than the original model ( $E_{org1}$ : CIFAR-10;  $E_{org2}$ : F-MNIST) when the percentage of the required fine inference for all input images is lower than 41.4% and 40.4% for (a) CIFAR-10 and (b) F-MNIST, respectively.

would be calculated by 20% multiplied by the total energy consumption (combining both the coarse model and the fine model) and adding it to 80% multiplied by only the coarse model's energy consumption. Thus, the energy consumption of the proposed approach ( $E_{pro}$ ) can be formulated as the following equation:

$$E_{pro} = (100 - N_F)\% \times E_{Coa} + N_F\% \times E_{CF}, \quad (1)$$

where  $N_F$ ,  $E_{Coa}$  and  $E_{CF}$  are the fine inference active time in percentage, the energy consumption of the coarse model and the energy consumption of the C2F model, respectively. Due to the C2F model consuming 0.384 W for the CIFAR-10 and 0.361 W for the F-MNIST dataset when the fine model is active, the energy consumption of CIFAR-10 and F-MNIST model ( $E_{CF}$ ) is  $4.4 \text{ ms} \times W = 1.69 \text{ mJ}$  and  $4.4 \text{ ms} \times 0.361 = 1.59 \text{ mJ}$ , respectively. Additionally, according to Table 5,  $E_{Coa}$  is equal to 0.888 and 0.835 mJ for CIFAR-10 and F-MNIST datasets, respectively. Therefore, if  $E_{pro1}$  is the energy consumption of the CIFAR-10 proposed model and  $E_{pro2}$  is for the F-MNIST model, their equations can be written based on Equation (1) as follows:

$$E_{pro1} = (100 - N_F)\% \times 0.888 + N_F\% \times 1.69, \quad (2)$$

$$E_{pro2} = (100 - N_F)\% \times 0.835 + N_F\% \times 1.59. \quad (3)$$

To find the position where the energy consumption of the proposed approach is lower than the original model,  $E_{pro}$  from Equations (2) and (3) are plotted against the constant line of the original model's energy consumption ( $E_{org}$ ) in Figure 13a,b. It is observed that when the percentage of fine model utilisation is less than 41.4% and 40.4% for the CIFAR-10 and F-MNIST input images, the energy consumption of the proposed method starts to be lower than the original model as shown in Figure 14. Moreover, the inference time of the C2F model begins to reduce w.r.t. original model when the activation percentage of the fine model is below 45.4% as shown in Figure 15. For example, in the case of the F-MNIST model for 20% of the inference processes (i.e. the fine model is activated), the energy saving was up to 13.5%, and the inference time is boosted by up to 7.4% compared to the original model. In the extreme case scenario, the input images are all out of our objective category of interest. This can result in a reduction of energy consumption by 27.2% and 26.7% for CIFAR-10 and F-MNIST dataset, respectively, and a reduction of inference time by 13.2% for both datasets. The threshold of 41.4% or 40.40% for fine model utilisation is favourable as in most real-world

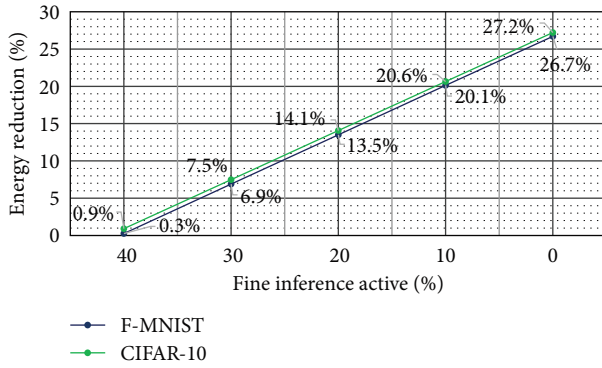


FIGURE 14: Energy reduction in percentage compared to the original model in the range where fine inference activation is less than 41.4% and 40.4% for CIFAR-10 and F-MNIST, respectively.

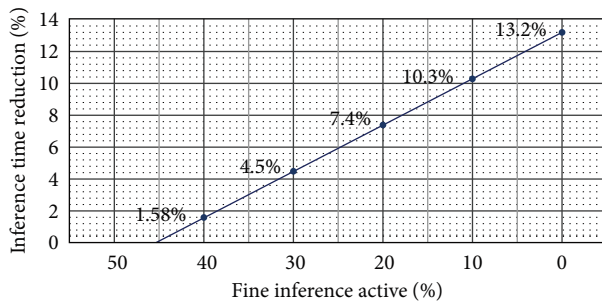


FIGURE 15: Inference time reduction in percentage compared to the original model in the range where fine inference activation is less than 45.4%.

scenarios the ratio of the items of interest vs. all items is far lower than this threshold, providing further gains as this ratio gets smaller. For example in CIFAR-10 dataset, our objective could be to classify all of the animals in a real environment which has far more number of vehicles compared to the animals. Using our approach by choosing the default fine model for animal classification can be really beneficial in term of energy consumption and inference time as the portion of the interested category is limited. Besides, in the case that the mission of the system changes, the fine inference model can be rapidly reprogrammed through PR from the external memory instead of downloading the full bit stream from the host computer.

## 6. Conclusion and Future Work

This article presents a FPGA-based design approach that relies on the use of dynamic and PR to offer high flexibility to perform CNN inference in real time on a single edge device considering different operation modes. The proposed approach uses PR to keep multiple fine models and the deeper layers of the original model in the external memory and activate only one of them at a time to minimise resource utilisation. Hence, the inference model can be reconfigured into the original model for typical classification in operation mode 1. Moreover, in operation mode 2, the inference

process begins with coarsely classifying the input images into superclasses first. Then, a fine model which is trained to recognise the image in a detailed class category is executed. Since the fine models are trained separately using particular datasets, they can achieve higher accuracy compared to the traditional models that are trained for a broad classification. A use case can be the mission of identifying limited fine categories. Due to the ability of coarse classification, the input images can be discarded using the shallow coarse model if they are not in the assigned coarse category. For instance, with the F-MNIST dataset, if we need to identify the object only in the 'outerwear' category, which constitutes only 20% of total images, the energy consumption and inference time can be reduced by 13.5% and 7.4%, respectively. This is advantageous for systems with limited resources.

The proposed method can be applied to a wide range of scientific domains. For example, in an anomaly detection application, the coarse model could be used to detect the abnormal activity, activating the fine model only upon detection to identify the type of anomaly. Such an approach can significantly reduce power consumption and inference time, given the anomaly activity rarely occurs.

Although the proposed approach mainly targets resource-constrained devices, it could be scaled up to address more complicated tasks. Instead of having two stages of coarse and fine models, the design could be extended in a hierarchical manner with more stages and models. Using this format, any early break during the process would result in huge savings in time, energy and resources. The scalability and flexibility of the proposed method make it easily applicable to the real-world applications and datasets.

Finally, our future plans include further investigation into preserving accuracy when implementing the approach with larger datasets, which may require a higher number of coarse categories. This investigation will involve exploring higher bit-widths and DSP utilisation as SIMD engines to maximise model performance.

## Data Availability Statement

The data that support the findings of this study are available from the corresponding author upon reasonable request.

## Conflicts of Interest

The authors declare no conflicts of interest.

## Funding

This research was performed as part of the employment of the authors at Loughborough University. Open access funding was enabled and organised by JISC.

## Acknowledgments

This research was performed as part of the employment of the authors at Loughborough University. Open access funding was enabled and organised by JISC.

## References

- [1] Z. Li, F. Liu, W. Yang, S. Peng, and J. Zhou, "A Survey of Convolutional Neural Networks: Analysis, Applications, and Prospects," *IEEE Transactions on Neural Networks and Learning Systems* 33, no. 12 (2021): 6999–7019.
- [2] D. Bhatt, C. Patel, H. Talsania, et al., "CNN Variants for Computer Vision: History, Architecture, Application, Challenges and Future Scope," *Electronics* 10, no. 20 (2021): 2470.
- [3] L. Chen, S. Li, Q. Bai, J. Yang, S. Jiang, and Y. Miao, "Review of Image Classification Algorithms Based on Convolutional Neural Networks," *Remote Sensing* 13, no. 22 (2021): 4712.
- [4] S. S. A. Zaidi, M. S. Ansari, A. Aslam, N. Kanwal, M. Asghar, and B. Lee, "A Survey of Modern Deep Learning Based Object Detection Models," *Digital Signal Processing* 126 (2022): 103514.
- [5] P. Vallimeena, U. Gopalakrishnan, B. B. Nair, and S. N. Rao, "CNN Algorithms for Detection of Human Face Attributes-A Survey," in *2019 International Conference on Intelligent Computing and Control Systems (ICCS)*, (IEEE, 2019): 576–581.
- [6] M. M. H. Shuvo, S. K. Islam, J. Cheng, and B. I. Morshed, "Efficient Acceleration of Deep Learning Inference on Resource-Constrained Edge Devices: A Review," *Proceedings of the IEEE* 111, no. 1 (2022): 42–91.
- [7] J. Chen and X. Ran, "Deep Learning With Edge Computing: A Review," *Proceedings of the IEEE* 107, no. 8 (2019): 1655–1674.
- [8] S. Mittal, "A Survey of FPGA-Based Accelerators for Convolutional Neural Networks," *Neural Computing and Applications* 32, no. 4 (2020): 1109–1139.
- [9] P. Dhilleswararao, S. Boppu, M. S. Manikandan, and L. R. Cenkeramaddi, "Efficient Hardware Architectures for Accelerating Deep Neural Networks: Survey," *IEEE Access* 10 (2022): 131788–131828.
- [10] J. Park and W. Sung, "FPGA Based Implementation of Deep Neural Networks Using On-Chip Memory Only," in *2016 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, (IEEE, 2016): 1011–1015.
- [11] M. Zhang, L. Li, H. Wang, Y. Liu, H. Qin, and W. Zhao, "Optimized Compression for Implementing Convolutional Neural Networks on FPGA," *Electronics* 8, no. 3 (2019): 295.
- [12] A. J. Abd.El.Maksoud, A. Mohamed, A. Tarek, et al., "FPGA Design of High-Speed Convolutional Neural Network Hardware Accelerator," in *2021 3rd Novel Intelligent and Leading Emerging Sciences Conference (NILES)*, (IEEE, 2021): 376–379.
- [13] R. Clarke and L. B. Moses, "The Regulation of Civilian Drones' Impacts on Public Safety," *Computer Law & Security Review* 30, no. 3 (2014): 263–285.
- [14] S. Carter, T. Johnston, S. Lidel, R. Riedel, and L. Tusch, "Drone Delivery: More Lift than You Think," <https://www.mckinsey.com/industries/aerospace-and-defense/our-insights/future-air-mobility-blog/drone-delivery-more-lift-than-you-think>.
- [15] M. Bojarski, D. D. Testa, D. Dworakowski, et al., "End to End Learning for Self-Driving Cars," (2016).
- [16] D. Gandhi, L. Pinto, and A. Gupta, "Learning to Fly by Crashing," (2017).
- [17] Y. Seo and Ks Shin, "Hierarchical Convolutional Neural Networks for Fashion Image Classification," *Expert Systems with Applications* 116 (2019): 328–339.
- [18] H. H. Pham, T. T. Le, D. Q. Tran, D. T. Ngo, and H. Q. Nguyen, "Interpreting Chest X-Rays via CNNs That Exploit Hierarchical Disease Dependencies and Uncertainty Labels," *Neurocomputing* 437 (2021): 186–194.
- [19] M. O. Turkoglu, S. D'Aronco, G. Perich, et al., "Crop Mapping From Image Time Series: Deep Learning With Multi-Scale Label Hierarchies," *Remote Sensing of Environment* 264 (2021): 112603.
- [20] K. Vipin and S. A. Fahmy, "ZyCAP: Efficient Partial Reconfiguration Management on the Xilinx Zynq," *IEEE Embedded Systems Letters* 6, no. 3 (2014): 41–44.
- [21] P. Plagwitz, F. Hannig, M. Ströbel, C. Strohmeier, and J. Teich, "A Safari Through FPGA-Based Neural Network Compilation and Design Automation Flows," in *2021 IEEE 29th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, (IEEE, 2021): 10–19.
- [22] R. Collobert, K. Kavukcuoglu, and C. Farabet, "Torch7: A Matlab-Like Environment for Machine Learning" In: BigLearn, NIPS workshop. CONF (2011).
- [23] Y. Jia, E. Shelhamer, J. Donahue, et al., "Caffe: Convolutional Architecture for Fast Feature Embedding," in *Proceedings of the 22nd ACM International Conference on Multimedia*, (ACM, 2014): 675–678.
- [24] F. Chollet, "Keras," 2015, <https://keras.io>.
- [25] A. Paszke, S. Gross, F. Massa, et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems*, 32, (Curran Associates, Inc., 2019).
- [26] AMD Xilinx Inc., "Xilinx Vitis AI," 2023, <https://github.com/Xilinx/Vitis-AI>.
- [27] K. Vissers, "FINN: A Framework for Fast, Scalable Binarized Neural Network Inference," in *Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, (ACM, 2017): 65–74.
- [28] M. Blott, T. B. Preußner, N. J. Fraser, et al., "FINN-R: An End-to-End Deep-Learning Framework for Fast Exploration of Quantized Neural Networks," *ACM Transactions on Reconfigurable Technology and Systems (TRETS)* 11, no. 3 (2018): 1–23.
- [29] J. Duarte, S. Han, P. Harris, et al., "Fast Inference of Deep Neural Networks in FPGAs for Particle Physics," *Journal of Instrumentation* 13, no. 7 (2018): P07027.
- [30] S. I. Venieris and C. S. Bouganis, "fpgaConvNet: A Framework for Mapping Convolutional Neural Networks on FPGAs," in *IEEE 24th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, (IEEE, 2016): 40–47.
- [31] H. Sharma, J. Park, D. Mahajan, et al., "From High-Level Deep Neural Models to FPGAs," in *49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, (IEEE, 2016): 1–12.
- [32] K. Abdelouahab, M. Pelcat, J. Serot, C. Bourrasset, and F. Berry, "Tactics to Directly Map CNN Graphs on Embedded FPGAs," *IEEE Embedded Systems Letters* 9, no. 4 (2017): 113–116.
- [33] A. Pappalardo, "Xilinx/Brevitas," (2023).
- [34] AMD Xilinx Inc., "Xilinx Vivado," 2023, <https://www.xilinx.com/products/design-tools/vivado.html>.
- [35] ARM, "AMBA Specifications," 2023, <https://www.arm.com/architecture/system-architectures/amba/amba-specifications>.
- [36] AMD Xilinx Inc., "Vivado Design Suite User Guide: Dynamic Function," 2023, <https://docs.xilinx.com/r/2022.2-English/ug909-vivado-partial-reconfiguration/Introduction>.
- [37] AMD Xilinx Inc., "Zynq 7000 SoC Technical Reference Manual (ug585)," 2023, <https://docs.xilinx.com/r/en-US/ug585-zynq-7000-SoC-TRM/PCAP-Throughput>.
- [38] M. Andreetto and H. Adam, "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," arXiv preprint arXiv: 1704.04861 (2017).

- [39] N. Ma, X. Zhang, H. T. Zheng, and J. Sun, "ShuffleNet V2: Practical Guidelines for Efficient CNN Architecture Design," in *Proceedings of the European Conference on Computer Vision (ECCV)*, (CVF, 2018): 116–131.
- [40] S. Bouguezzi, H. B. Fredj, T. Belabed, C. Valderrama, H. Faiedh, and C. Souani, "An Efficient FPGA-Based Convolutional Neural Network for Classification: Ad-MobileNet," *Electronics* 10, no. 18: 2272.
- [41] Z. Fan, W. Hu, H. Guo, F. Liu, and D. Xu, "Hardware and Algorithm Co-Optimization for Pointwise Convolution and Channel Shuffle in ShuffleNet V2," in *IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, (Melbourne, Australia: IEEE, 2021), 3212–3217.
- [42] W. Pang, C. Wu, and S. Lu, "An Energy-Efficient Implementation of Group Pruned CNNs on FPGA," *IEEE Access* 8 (2020): 217033–217044.
- [43] L. Liu and S. Brown, "Leveraging fine-grained structured sparsity for CNN inference on systolic array architectures," in *2021 31st International Conference on Field-Programmable Logic and Applications (FPL)*, (IEEE, 2021): 301–305.
- [44] T. Yang, Y. Liao, J. Shi, Y. Liang, N. Jing, and L. Jiang, "A Winograd-Based CNN Accelerator With a Fine-Grained Regular Sparsity Pattern," in *30th International Conference on Field-Programmable Logic and Applications (FPL)*, (IEEE, 2020): 254–261.
- [45] D. T. Nguyen, H. Kim, and H. J. Lee, "Layer-Specific Optimization for Mixed Data Flow with Mixed Precision in FPGA Design for CNN-Based Object Detectors," *IEEE Transactions on Circuits and Systems for Video Technology* 31, no. 6 (2020): 2450–2464.
- [46] X. Sui, Q. Lv, Y. Bai, et al., "A Hardware-Friendly Low-Bit Power-of-Two Quantization Method for CNNs and Its FPGA Implementation," *Sensors* 22, no. 17 (2022): 6618.
- [47] S. Liang, S. Yin, L. Liu, W. Luk, and S. Wei, "FP-BNN: Binarized Neural Network on FPGA," *Neurocomputing* 275 (2018): 1072–1086.
- [48] M. Farhadi, M. Ghasemi, and Y. Yang, "A Novel Design of Adaptive and Hierarchical Convolutional Neural Networks Using Partial Reconfiguration on FPGA," in *2019 IEEE High Performance Extreme Computing Conference (HPEC)*, (IEEE, 2019): 1–7.
- [49] B. Seyoum, M. Pagani, A. Biondi, S. Balleri, and G. Buttazzo, "Spatio-Temporal Optimization of Deep Neural Networks for Reconfigurable FPGA SoCs," *IEEE Transactions on Computers* 70, no. 11 (2021): 1988–2000.
- [50] D. Cain, O. Eldash, K. Khalil, and M. Bayoumi, "Convolution Processing Unit Featuring Adaptive Precision Using Dynamic Reconfiguration," in *2021 IEEE 7th World Forum on Internet of Things (WF-IoT)*, (IEEE, 2021): 592–597.
- [51] H. Irmak, F. Corradi, P. Detterer, N. Alachiotis, and D. Ziener, "A Dynamic Reconfigurable Architecture for Hybrid Spiking and Convolutional FPGA-Based Neural Network Designs," *Journal of Low Power Electronics and Applications* 11, no. 3 (2021): 32.
- [52] L. Deng, "The MNIST Database of Handwritten Digit Images for Machine Learning Research," *IEEE Signal Processing Magazine* 29, no. 6 (2012): 141–142.
- [53] G. K. Cohen, S. Afshar, J. Tapson, and A. van Schaik, "EMNIST: An Extension of MNIST to Handwritten Letters," Arxiv preprint (2017).
- [54] E. Youssef, H. A. Elsimary, M. A. El-Moursy, H. Mostafa, and A. Khattab, "Energy-Efficient Precision-Scaled CNN Implementation With Dynamic Partial Reconfiguration," *IEEE Access* 10 (2022): 95571–95584.
- [55] A. Krizhevsky and G. Hinton, "Learning Multiple Layers of Features From Tiny Images," (2009).
- [56] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: A Novel Image Dataset for Benchmarking Machine Learning Algorithms," (2017).
- [57] Z. Yan, H. Zhang, R. Piramuthu, et al., "HD-CNN: Hierarchical Deep Convolutional Neural Networks for Large Scale Visual Recognition," in *Proceedings of the IEEE International Conference on Computer Vision*, (IEEE, 2015): 2740–2748.
- [58] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," arXiv preprint arXiv: 14091556 (2014).
- [59] S. Ioffe and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," in *Proceedings of the 32nd International Conference on Machine Learning*, (PMLR, 2015): 448–456.